

# Table des matières

---

|                                                                 |   |
|-----------------------------------------------------------------|---|
| 1. Thèse et objet du protocole                                  | 1 |
| 2. Ligne de base contrôlée : l'erreur à ne pas répéter          | 2 |
| 2.1 Deux livrables, deux échecs mesurés . . . . .               | 2 |
| 2.2 La cause racine commune . . . . .                           | 3 |
| 3. Les leviers d'apprentissage                                  | 3 |
| 3.1 EVO-000020 — YAHRIA sait se mesurer . . . . .               | 3 |
| 3.2 EVO-000016 — apprendre à ne plus répéter . . . . .          | 3 |
| 4. La porte de boot exécutable et son calibrage                 | 4 |
| 4.1 Sept étapes, de l'inventaire à l'OpenAPI . . . . .          | 4 |
| 4.2 Rétro-test de calibrage . . . . .                           | 5 |
| 4.3 Armement gouverné . . . . .                                 | 6 |
| 5. Déroulé du test décisif                                      | 6 |
| 5.1 Phases, acteurs et preuves . . . . .                        | 7 |
| 5.2 Le RUN-000022 : un test équitable du même domaine . . . . . | 7 |
| 5.3 Exécution outillée . . . . .                                | 8 |
| 6. Critères de verdict et gouvernance                           | 8 |
| 6.1 Matrice de verdict . . . . .                                | 8 |
| 6.2 Invariants invoqués . . . . .                               | 9 |
| 6.3 Frontières assumées . . . . .                               | 9 |

# 1. Thèse et objet du protocole

Ce document définit le protocole PTA-001, test décisif de l'apprentissage pour la plateforme YAHRIA CODE OS. Sa thèse de départ tient en une phrase, formulée par l'opérateur : YAHRIA a appris à se mesurer, il n'a pas encore appris à ne plus répéter son erreur. Depuis les sessions R7 à R21, la plateforme s'est dotée d'une considérable capacité de mesure — registre d'évolution avec pipeline à sept états, preuves scellées chaînées, audits de complétude, suites de régression — mais cette capacité mesure l'erreur après coup. Le protocole PTA-001 transforme la mesure en prévention : une décision humaine dans le registre d'évolution doit modifier mécaniquement le comportement du générateur, et un nouveau livrable doit le démontrer en exécution réelle.

La distinction est essentielle entre un apprentissage déclaré et un apprentissage prouvé. Un apprentissage déclaré est un texte : un rapport, une proposition, une note de version. Un apprentissage prouvé est un fait ré-exécutable : le système se comporte différemment après la décision qu'avant, et cette différence se mesure sur un cas concret, sans intervention corrective humaine. C'est pourquoi le protocole exige un critère binaire et public : le RUN-000022, généré par le pipeline sous la nouvelle règle, doit démarrer au premier coup. S'il démarre, l'apprentissage est prouvé ; s'il ne démarre pas, l'échec est mesuré, scellé, et le cycle d'évolution repart — l'apprentissage n'est pas déclaré, il est en cours.

Le protocole repose sur trois objets précis. Premièrement, la proposition EVO-000016 « prévalidation boot avant SEALED », actuellement en UNDER\_REVIEW : son approbation et sa promotion par l'autorité humaine arment une porte de boot exécutable dans le pipeline du studio. Deuxièmement, la proposition EVO-000020, déjà PROMOTED : elle prouve que la boucle de gouvernance fonctionne de bout en bout et sert de levier de référence. Troisièmement, le RUN-000022, une génération neuve dans le même domaine que l'erreur historique — un hub de paiement Mobile Money en FastAPI — dont le premier démarrage constitue le verdict. Chaque objet est adossé à un artefact ré-exécutable : script de décision, jumeau de porte, orchestrateur de test, rapports JSON horodatés.

|                                                                                              |                                                                                       |                                                                                 |                                                                                         |
|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <div>0/2</div> <div>taux de boot au premier coup — historique RUN-000019 et RUN-000021</div> | <div>44</div> <div>flots non importables détectés en 6 ms par la porte calibrée</div> | <div>7</div> <div>étapes de la porte de boot, de l'inventaire à l'OpenAPI</div> | <div>1</div> <div>décision humaine arme la porte — le registre est l'interrupteur</div> |
|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|

Indicateurs clés du protocole à la date d'émission

## 2. Ligne de base contrôlée : l'erreur à ne pas répéter

### 2.1 Deux livrables, deux échecs mesurés

La ligne de base du protocole est constituée par les deux livrables qui ont échoué leur premier démarrage, avec pour chacun des chiffres mesurés en exécution réelle et non estimés. Le livrable RUN-000019, logiciel de gestion hôtelière en Express et Mongoose, était scellé par le studio après vérification statique, puis crashait instantanément au premier démarrage : un middleware objet passé comme fonction, une variable d'environnement divergente qui figeait le processus, du SQL PostgreSQL contre un modèle Mongoose, une route de connexion jamais montée, un second point d'entrée cassé, un nommage incohérent entre validateurs et modèles. Six défauts structurels, aucun détecté par la vérification statique du studio au moment du scellement.

Le livrable RUN-000021, ERP de chaîne hôtelière avec hub API Mobile Money 8.3, a aggravé le diagnostic sur toutes les dimensions. L'audit statique a compté douze fichiers sur quarante-quatre syntaxiquement cassés par troncature, soit vingt-sept pour cent du livrable, et zéro import croisé : trente-trois îlots sans aucune couture entre eux. Les onze fichiers Python du hub, théoriquement le cœur de la livraison, étaient enfouis sous des arborescences de dossiers aux noms comportant espaces, accents et émojis — des chemins de packages qu'aucun import Python ne peut jamais résoudre. La preuve par exécution a établi une cascade de cinq échecs successifs, depuis les imports relatifs illégaux jusqu'aux symboles hallucinés, et un seul îlot bootable sur trente-trois. Le tableau suivant consolide la ligne de base mesurée.

| Livrable                       | Défaut mesuré                                                                                                      | Résultat au premier boot                           | Scellé quand même                           |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|---------------------------------------------|
| RUN-000019 — Express/Mongoose  | 6 défauts structurels (middleware objet, MONGO_URI, SQL/PostgreSQL, login non routé, index.js cassé, nommage)      | Crash instantané — TypeError au lancement          | Oui — vérification statique seule (INV-215) |
| RUN-000021 — ERP + Hub API 8.3 | 12/44 fichiers tronqués (27%), 0 import croisé, 44 îlots non importables, cascade de 5 échecs, symboles hallucinés | Aucun assemblage possible — 1 îlot bootable sur 33 | Oui — livré en l'état                       |

Tableau 1 — Ligne de base mesurée : les deux échecs du premier boot

## 2.2 La cause racine commune

La cause racine est commune aux deux cas et elle est structurelle, pas accidentelle : le scellement intervient après une vérification statique qui contrôle des signes de complétude — absence de placeholders, équilibrage des délimiteurs, validité JSON, taille — mais jamais le fait que le logiciel démarre. L'invariant INV-215 documente cette frontière honnête : les livrables sont scellés sans boot réel. Tant que cette frontière demeure, la plateforme est structurellement condamnée à reproduire l'erreur, quelle que soit la qualité de ses mesures a posteriori. Le protocole PTA-001 a donc pour fonction de déplacer la vérification du boot en amont du scellement, et de faire de ce déplacement une décision gouvernée plutôt qu'un correctif local.

## 3. Les leviers d'apprentissage

---

### 3.1 EVO-000020 — YAHRIA sait se mesurer

Le premier levier est EVO-000020, déjà PROMOTED : elle établit que la boucle de gouvernance exécute réellement son cycle. Cette proposition a traversé les sept états du pipeline — DRAFTED, SUBMITTED, UNDER\_REVIEW, APPROVED par l'autorité humaine via l'interface, SCHEDULED par le pipeline, PROMOTED avec une expérimentation documentée au sens de l'INV-162 et un plan de rollback au sens de l'INV-163 — sous la preuve scellée EV-POLICY-000531. Son expérimentation compare la décomposition réelle de la mission MIS-000017, trois tâches génériques sans critère de preuve, à une re-décomposition miroir à cinq étapes couvrant six défauts sur six. Elle démontre que YAHRIA mesure : elle ne démontre pas encore qu'il change de comportement.

### 3.2 EVO-000016 — apprendre à ne plus répéter

Le second levier est EVO-000016 « Sandbox Docker pour prévalidation npm (INV-215) », actuellement UNDER\_REVIEW, que l'opérateur reformule exactement : prévalidation du boot avant le scellement. Sa promotion doit produire un changement mécanique, pas un engagement. C'est pourquoi le protocole installe l'armement dans le code même du pipeline : le module boot-gate.ts lit le registre d'évolution à chaque exécution et n'active la porte que si EVO-000016 est à l'état PROMOTED. La décision humaine devient ainsi l'interrupteur réel du comportement système — aucune variable d'environnement, aucun contournement, aucun geste manuel. Tant que la proposition est en révision, la porte est inerte et la frontière INV-215 demeure, documentée et honnête.

| Levier                                       | État                    | Ce que le levier démontre                                                             | Preuve                                           |
|----------------------------------------------|-------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------|
| EVO-000020 — décomposition granulaire D.07   | PROMOTED                | La boucle gouvernée fonctionne : mesure, décision humaine, expérimentation, promotion | EV-POLICY-000531, cycle 7 états complet          |
| EVO-000016 — prévalidation boot avant SEALED | UNDER_REVIEW → décision | La décision humaine modifie mécaniquement le comportement du générateur               | boot-gate.ts lit le registre ; calibrage PTA-001 |

Tableau 2 — Les deux leviers du test d'apprentissage

## 4. La porte de boot exécutable et son calibrage

### 4.1 Sept étapes, de l'inventaire à l'OpenAPI

La porte de boot est l'implémentation exécutable d'EVO-000016. Elle existe en deux jumeaux strictement équivalents : une intégration TypeScript dans le pipeline du studio, qui bloque la transition vers SEALED en cas d'échec, et un vérificateur indépendant en Python, qui re-juge tout livrable — répertoire ou archive ZIP — hors du pipeline. Sept étapes la composent, dans un ordre calculé pour échouer le plus tôt possible sur la cause la plus probable. Les étapes STRUCTURE et SYNTAXE encadrent exactement les dégâts observés sur RUN-000021 : îlots de code hors module et fichiers tronqués. Les étapes BOOT, SONDES et OPENAPI mesurent ce que la vérification statique ne pourra jamais mesurer : le fait que le logiciel démarre réellement, répond sur un port, expose ses routes.

| Étape         | Ce qu'elle contrôle                                              | Échec détecté (exemples réels)                               |
|---------------|------------------------------------------------------------------|--------------------------------------------------------------|
| 1. INVENTAIRE | Composition du livrable par langage, arborescence                | livrable vide, stack indéterminable                          |
| 2. STRUCTURE  | Importabilité : extensions .py, identifiants de packages valides | 44 îlots de RUN-000021 (code hors module, chemins invalides) |
| 3. SYNTAXE    | Compilation Python de chaque fichier                             | 12 troncatures de RUN-000021 (27%)                           |
| 4. DÉCOUVERTE | Instance FastAPI/create_app trouvable et importable              | symboles hallucinés, double montage                          |
| 5. BOOT       | Démarrage uvicorn réel sur port libre, fenêtre 30 s              | cascades d'imports, blocages au chargement                   |
| 6. SONDES     | Réponses HTTP réelles sur 4 routes de référence                  | processus sorti, connexion refusée                           |
| 7. OPENAPI    | Décodage de la spécification et décompte des routes              | spécification absente ou vide                                |

Tableau 3 — Les sept étapes de la porte de boot

## 4.2 Rétro-test de calibrage

Une porte qui n'aurait jamais vu l'erreur ne prouve rien ; une porte qui refuse un livrable sain détruirait plus qu'elle ne protège. Le protocole impose donc un calibrage par rétro-test sur les deux livrables réels, avant toute promotion. Le résultat est conforme des deux côtés. Sur le livrable original RUN-000021, la porte échoue en six millisecondes dès l'étape **STRUCTURE** avec quarante-quatre îlots non importables identifiés — cent pour cent du livrable, ce qui correspond exactement au verdict de l'audit : zéro pour cent assemblable. Sur le livrable corrigé, elle passe les sept étapes en deux virgule cinq secondes, avec un démarrage uvicorn réel de deux secondes, des sondes HTTP vertes et une spécification OpenAPI décodée — zéro faux positif. Cette double conformité constitue l'expérimentation documentée exigée par l'INV-162 pour la promotion.

La figure suivante présente la durée réelle de chaque étape sur le livrable sain. Elle illustre le coût du contrôle : environ deux secondes et demie au total, dont l'essentiel est le boot lui-même — précisément la preuve que la plateforme ne produisait jamais. Sur le livrable défaillant, la porte s'arrête six millisecondes après l'inventaire : le livrable mort est écarté avant même de coûter un démarrage. Le rapport complet des deux exécutions est archivé en JSON horodaté sous `scripts/pta-evidence/calibration-original.json` et `calibration-corrige.json`, et chaque exécution de la porte scelle sa propre preuve dans le registre d'évidence de la plateforme, conformément à l'INV-110.

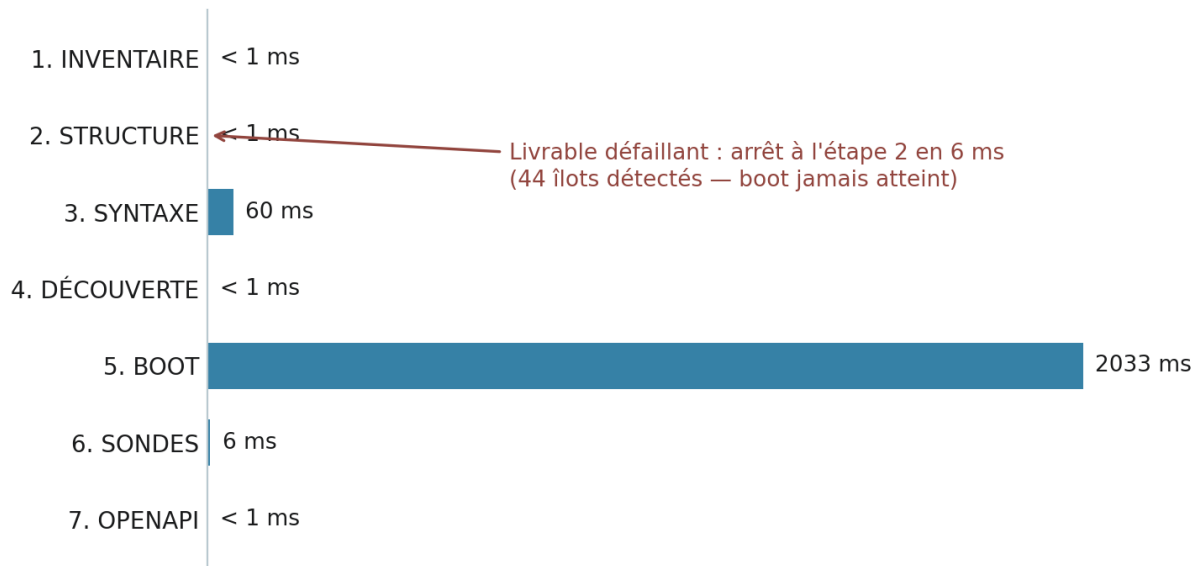


Figure 1 — Durée mesurée de chaque étape de la porte sur le livrable sain (RUN-000021-corrige, PASS en 2 473 ms). Le livrable défaillant s'arrête à l'étape STRUCTURE en 6 ms.

### 4.3 Armement gouverné

L'armement est gouverné par construction. En l'état actuel — EVO-000016 en UNDER\_REVIEW — la porte intégrée au pipeline est inertée par sa propre logique de lecture du registre : elle s'exécute à chaque run, constate l'absence de promotion, et ne bloque rien. La séquence d'approbation, de programmation et de promotion est préparée dans le pack de décision pta-evo16-approve.sh, avec le plan de rollback rédigé au sens de l'INV-163 et l'expérimentation de calibrage documentée au sens de l'INV-162. L'INV-228 est respecté dans son esprit comme dans sa lettre : la promotion n'exécute aucune mutation de production — elle change l'état d'un registre ; c'est la lecture de cet état par le pipeline qui arme la porte au run suivant. Ce montage fait de la décision humaine la cause efficiente du changement de comportement, exactement ce que le test d'apprentissage entend prouver.

## 5. Déroulé du test décisif

## 5.1 Phases, acteurs et preuves

Le protocole se déroule en sept phases, chacune avec un acteur nommé et une preuve attendue. Les phases P0 et P1 sont d'ores et déjà terminées : la ligne de base est mesurée et scellée, la porte est construite et calibrée. La phase P2 appartient exclusivement à l'autorité humaine : approuver, programmer et promouvoir EVO-000016 via l'interface de l'onglet évolution — le promoteur de la proposition étant l'agent super-z, l'INV-227 interdit mécaniquement qu'il approuve sa propre proposition. Les phases P3 à P6 sont exécutées par l'agent sous la porte armée, sans intervention corrective : c'est cette absence de rattrapage humain qui donne au verdict sa valeur.

| Phase | Contenu                                                   | Acteur | Statut     | Preuve attendue                             |
|-------|-----------------------------------------------------------|--------|------------|---------------------------------------------|
| P0    | Ligne de base : audit RUN-000021, chiffres scellés        | Agent  | TERMINÉ    | audit 44 fichiers, preuves live             |
| P1    | Construction + calibrage de la porte (rétro-test)         | Agent  | TERMINÉ    | calibration-original/corrige.json           |
| P2    | Approbation, programmation, promotion EVO-000016          | Humain | EN ATTENTE | registre : PROMOTED, preuves 3 états        |
| P3    | Armement vérifié : la porte lit le registre               | Agent  | PRÊT       | boot-gate.ts, tsc 0 erreur                  |
| P4    | Génération RUN-000022 sous porte armée                    | Agent  | PRÊT       | run Studio PYTHON, brief porteur des leçons |
| P5    | Mesure : porte interne + vérification indépendante du ZIP | Agent  | PRÊT       | pta-run22.sh, verdict JSON                  |
| P6    | Verdict L-PROVEN / L-NOT-PROVEN + archivage               | Agent  | PRÊT       | run22-verdict.json, mission de trace        |

Tableau 4 — Les sept phases du protocole

## 5.2 Le RUN-000022 : un test équitable du même domaine

Le RUN-000022 est conçu comme un test équitable du même domaine que l'erreur, pas de sa reproduction intégrale. Reproduire les quarante-quatre fichiers de l'ERP garantirait l'échec pour des raisons de budget de génération, indépendantes de tout apprentissage ; le protocole fixe donc un périmètre comparable et atteignable : un hub de paiement Mobile Money en FastAPI, deux passerelles derrière une factory, des webhooks signés en HMAC-SHA256 à comparaison de temps constant, du stockage en mémoire, des tests. Le brief intègre explicitement les leçons de la ligne de base : instance app nommée dans main.py, aucun serveur embarqué au niveau module, valeurs par défaut pour toutes les variables d'environnement, dossiers en identifiants Python valides, extension .py sur chaque fichier de code, requirements.txt déclaré. Le test mesure précisément ceci : le pipeline honore-t-il ces leçons au premier essai ?



### 5.3 Exécution outillée

L'exécution est outillée de bout en bout. Le pack de décision pta-evo16-approve.sh propose le mode sec par défaut et n'exécute l'approbation, la programmation et la promotion que sur commande explicite de l'opérateur, avec l'expérimentation de calibrage en argument. L'orchestrateur pta-run22.sh refuse de démarrer si la porte n'est pas armée — un test lancé sans la règle ne prouverait rien — puis crée le run Studio en imposant la stack PYTHON et l'arborescence, suit le pipeline jusqu'à SEALED ou FAILED, télécharge l'archive scellée, la re-juge avec le vérificateur indépendant, et produit le verdict en JSON. Une mission de trace est créée dans le registre des missions, comme pour MIS-000017 et MIS-000018, afin que le protocole lui-même soit traçable par la plateforme qu'il teste.

## 6. Critères de verdict et gouvernance

### 6.1 Matrice de verdict

Le verdict est une matrice binaire, définie avant l'exécution et non négociable après. Trois conditions cumulatives définissent la preuve : le run atteint l'état SEALED au premier passage du pipeline, la porte intégrée rapporte PASS dans les statistiques du run, et le vérificateur indépendant rapporte PASS sur l'archive téléchargée. Une seule condition manquante bascule le verdict en L-NOT-PROVEN. Il n'existe pas de verdict intermédiaire, pas de réussite partielle, pas de seconde chance silencieuse : toute correction post-génération invaliderait précisément ce que le test mesure. Dans ce cas, l'échec est archivé comme preuve, ses causes sont diagnostiquées, et une nouvelle itération du cycle d'évolution D.15 est ouverte — la mesure de l'apprentissage continue, ce qui est exactement la fonction d'un protocole.

| Verdict                               | Conditions cumulatives                                                                             | Conséquence                                                                             |
|---------------------------------------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| L-PROVEN — apprentissage prouvé       | RUN-000022 SEALED au 1er passage ; porte interne PASS ; vérification indépendante PASS sur le ZIP  | EVO-000016 validée en conditions réelles ; l'apprentissage est un fait ré-exécutable    |
| L-NOT-PROVEN — apprentissage en cours | Au moins une condition manque (échec de porte, run FAILED, divergence du vérificateur indépendant) | Échec scellé comme preuve ; diagnostic ; nouvelle itération D.15 ; la porte reste armée |

Tableau 5 — Matrice de verdict, définie avant l'exécution

## 6.2 Invariants invoqués

Le protocole est gouverné par les invariants de la plateforme elle-même, qui en constituent la constitution du test. L'INV-227 garantit que la décision d'armement appartient à l'autorité humaine et non au proposeur. L'INV-163 exige un plan de rollback rédigé, fourni dans le pack de décision : retirer l'armement et re-marquer la proposition ROLLED\_BACK si la porte produisait des faux positifs en conditions réelles. L'INV-162 exige une expérimentation documentée pour la promotion — le rétro-test de calibrage la fournit avec ses rapports JSON horodatés. L'INV-110 scelle une preuve à chaque transition. L'INV-215, la frontière honnête que le protocole ferme, demeure documentée pour les stacks hors périmètre de la version un : la porte boot réellement les projets Python et vérifie la syntaxe des projets Node, la forme complète par sandbox Docker restant l'horizon d'EVO-000016 sur un hôte équipé.

| Invariant                                  | Rôle dans le protocole                                                | État                           |
|--------------------------------------------|-----------------------------------------------------------------------|--------------------------------|
| INV-227 — proposeur ≠ décideur             | L'agent super-z a proposé EVO-000016 ; l'autorité humaine approuve    | <b>Appliqué mécaniquement</b>  |
| INV-163 — rollback obligatoire             | Plan de rollback rédigé, $\geq 15$ caractères, prêt dans le pack      | <b>Préparé</b>                 |
| INV-162 — expérimentation documentée       | Rétro-test calibré : FAIL 6 ms sur l'original, PASS 2,5 s sur le sain | <b>Prouvé (rapports JSON)</b>  |
| INV-110 — preuve par transition            | Chaque état du pipeline et de la porte scelle sa preuve               | <b>Actif</b>                   |
| INV-215 — frontière du scellement statique | La frontière que le protocole ferme pour PYTHON ; documentée ailleurs | <b>Cible du test</b>           |
| INV-228 — PROMOTED n'exécute rien          | La promotion change un registre ; la porte lit ce registre            | <b>Respecté par conception</b> |

Tableau 6 — Constitution du test : les invariants appliqués

## 6.3 Frontières assumées

Le protocole assume enfin ses frontières, car un test qui exagérerait sa portée reproduirait le travers qu'il dénonce. La porte version un vérifie le boot réel pour la stack Python, qui est le domaine de l'erreur historique, et la syntaxe pour Node ; les autres stacks passent en SKIP documenté, pas en PASS trompeur. Le RUN-000022 teste un périmètre comparable et atteignable, pas la reproduction des quarante-quatre fichiers dont le budget de génération avait déjà condamné la forme. La décision P2 reste la propriété exclusive de l'opérateur : ce document prépare tout, il ne décide rien. Si la porte s'arme et que le premier run démarre, l'apprentissage sera prouvé, pas déclaré — et si le premier run échoue, la plateforme aura gagné quelque chose de plus rare qu'une réussite : la mesure exacte de ce qui lui reste à apprendre.